Постановка задачи

Требуется реализовать шаблонный односвязный список. В списке должны быть реализованы следующие методы:

1. Реализовать правило пяти: деструктор; конструктор копирования; оператор присваивания копированием; конструктор перемещения; оператор присваивания перемещением.
2. `void push_back(const T& value)` — добавляет элемент в конец списка;
3. `void push_front(const T& value)` — добавляет элемент в начало списка;
4. `void insert(size_t index, const T& value)` — добавляет элемент по указанному индексу (вставка перед элементом с данным индексом);
5. `void pop_back()` — удаляет последний элемент списка;
6. `void pop_front()` — удаляет первый элемент списка;
7. `void remove_at(size_t index)` — удаляет элемент списка по указанному индексу;
8. `T& operator[](size_t index)` — возвращает ссылку (для изменения значения);
9. `const T& operator[](size_t index) const` — возвращает константную ссылку на элемент по указанному индексу (для чтения);
10. `size_t size() const` — возвращает количество элементов в списке;
11. `bool empty() const` — возвращает `true`, если список пуст;
12. `void clear()` — очищает список, удаляя все узлы;
13. `const T& front() const` — возвращает первый элемент списка;
14. `const T& back() const` — возвращает последний элемент списка.

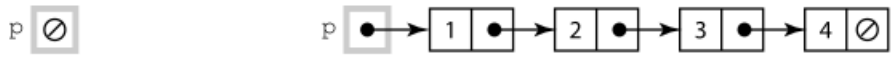
Где возможно, реализовать для списка класс `ListIterator` и добавить поддержку итераторов `begin()` и `end()`. Итератор должен поддерживать операторы: `*`; `++`; `!=`.

Проверить корректность работы итераторов с помощью тестового кода.

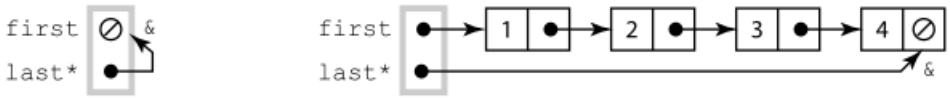
```
LList<int> l = { 3, 5, 2, 7, 8 }; int s = 0;
for (auto &i : l) s += i*10;
auto lambda = [](int a, int b) { return a + b*10; };
cout << s << '\t' << std::accumulate(l.begin(), l.end(), 0, lambda) << '\n';
```

Внутренне представление списка должно соответствовать одному из представленных вариантов. Необходимо выполнить только один вариант списка. Обратите внимание, что структура узла во всех вариантах идентична. В узле допустимо использование умных указателей. Использование внешних контейнеров запрещено. На рисунках представлен пустой (слева) и заполненный список (справа). При реализации используйте CppCoreGuidelines. Варианты:

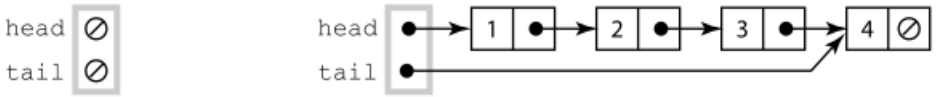
1. Указатель на первый узел, в последнем узле указатель на `nullptr`.



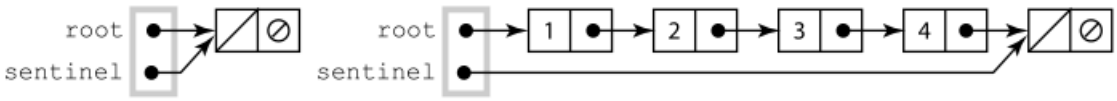
2. Указатель на первый узел. Указатель `last` является указателем на указатель `next`.



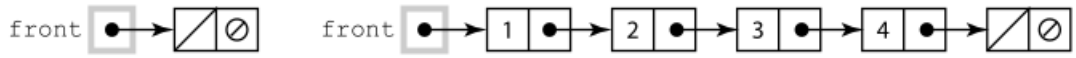
3. Указатель на первый и последний узел.



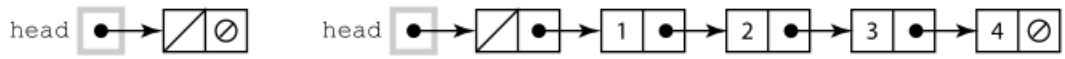
4. Два указателя, дополнительный служебный узел в конце списка.



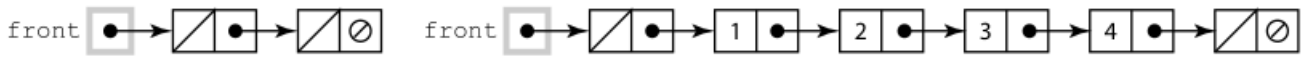
5. Дополнительный служебный узел в конце списка.



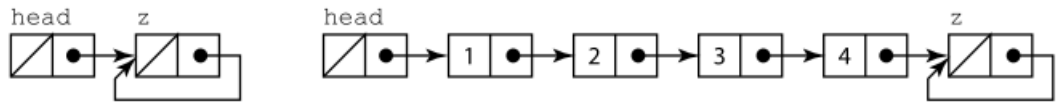
6. Дополнительный служебный узел в начале списка.



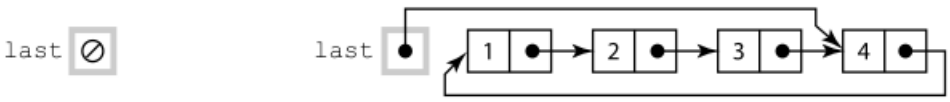
7. Два дополнительных служебных узла.



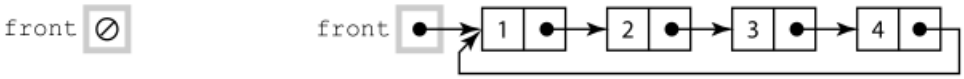
8. Дополнительный служебный узел указывает сам на себя.



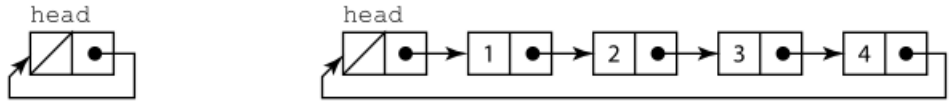
9. Циклический список, указатель на последний узел.



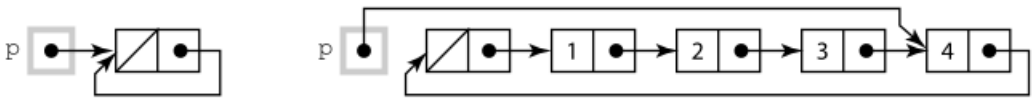
10. Циклический список, указатель на первый узел.



11. Циклический список с дополнительным служебным узлом.



12. Циклический список со служебным узлом, указатель на последний узел.



```cpp
void print_lst(const LList<char> &l) {
  // вывод списка на экран с помощью l[i]
}
int main() {
  LList<char> lst; // ваш список
  std::cout << std::boolalpha << lst.empty() << std::endl;

  for(int i = 0; i < 5; i++)
    lst.push_back(char('a' + i));

  print_lst(lst);

  for(int i = 0; i < 5; i++)
    lst.insert(0, char('z' - i));

  print_lst(lst);

  for(size_t i = 0; i != lst.size(); i++)
    lst[i] = char('a' + i);

  print_lst(lst);

  lst.pop_back();
  lst.pop_front();

  print_lst(lst);

  lst.remove_at(5);
  lst.insert(3, 'o');

  print_lst(lst);

  lst.clear();

  lst.push_back('q');
  lst.push_back('w');

  std::cout << lst.size() << ' ' << std::boolalpha << lst.empty() << std::endl;
} // деструктор освободит оставшиеся узлы
```

Результат работы

Пример с использованием всех реализованных методов должен работать без ошибок. Вывод должен соответствовать примеру ниже.

Вывод приложения

```
true
a -> b -> c -> d -> e
v -> w -> x -> y -> z -> a -> b -> c -> d -> e
a -> b -> c -> d -> e -> f -> g -> h -> i -> j
b -> c -> d -> e -> f -> g -> h -> i
b -> c -> d -> o -> e -> f -> h -> i
2 false
```

Добавьте проверку реализации правила пяти.

```
//
  LList<char> a;

  for(char c = 'a'; c <= 'e'; c++)
  a.push_back(c);

  LList<char> b = a;   // copy constructor
  b[0] = 'Z';

  print_lst(a);
  print_lst(b);
//
  LList<char> a;
  LList<char> b;

  for(char c = 'a'; c <= 'e'; c++)
  a.push_back(c);

  b = a;   // copy assignment

  b[1] = 'X';

  print_lst(a);
  print_lst(b);
//
  LList<char> a;

  for(char c = 'a'; c <= 'e'; c++)
  a.push_back(c);

  LList<char> b = std::move(a);

  print_lst(b);
  std::cout << "size a = " << a.size() << std::endl;
//
  LList<char> a;
  LList<char> b;

  for(char c = 'a'; c <= 'e'; c++)
    a.push_back(c);

  b = std::move(a);

  print_lst(b);
//
  LList<int> l;

  l.push_back(1);
  l.push_back(2);
  l.push_back(3);

  l.insert(0, 100);
  l.insert(l.size(), 200);

  print_lst(l);
```

Файл/файлы с исходным кодом добавьте себе в репозиторий.